

Capitolo 13 — Il Request Routing WCM

Pubblico	Lettori tecnici e non tecnici del WCM Process & Memory Book
Versione	FROZEN-13
Data master	2026-08-29
Stato	FROZEN
Source path	wcm/documentation/process-memory-book/ chapters/13_request_routing_wcm.md
Source SHA	9e181a5
Release	2026-08-31T22:48:24.896Z

GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.

Stato: FROZEN **Parte:** V — Da una richiesta alle regole applicabili **Scope:** WCM generale / domain-agnostic **Review:** Technical Review PASS / Human Comprehension Review PASS

13.0 Una richiesta non è ancora un piano d'azione

Nei capitoli precedenti abbiamo costruito il sistema con cui WCM trova e valuta la conoscenza.

Abbiamo visto il Knowledge Navigation Layer, INDEX-FIRST, il Progressive Retrieval e la Source Precedence.

Ora cambiamo punto di osservazione.

Non partiamo più da una fonte.

Partiamo da una richiesta.

Può essere una frase molto semplice:

| «Aggiorna questo documento.»

Oppure:

| «Continua il lavoro.»

Oppure ancora:

| «C'è un problema. Risolvilo.»

Per un essere umano queste frasi possono sembrare sufficienti. In un'organizzazione agantica, invece, non lo sono necessariamente.

Prima di agire WCM deve capire almeno:

- che cosa viene chiesto davvero;
- su quale oggetto o progetto;
- con quale goal;
- entro quale scope;
- con quale authority;
- se esiste già un workflow da riprendere;
- se l'azione è una normale RUN o modifica il metodo;
- quali processi e protocolli sono applicabili;

- quali capability servono;
- dove si trova il contesto minimo necessario;
- quale condizione autorizza l'esecuzione;
- dove il lavoro deve realmente fermarsi.

Questa trasformazione è il **Request Routing WCM**.

Request Routing è il percorso con cui WCM trasforma una richiesta in un contesto operativo sufficientemente definito da poter essere eseguito, ripreso, delegato o fermato in modo governato.

Non è un singolo algoritmo universale.

È una disciplina di instradamento che combina comprensione cognitiva, fonti autorevoli, workflow persistenti, processi, protocolli, capability e guard deterministici quando disponibili.

13.1 Arriva una richiesta: cosa succede?

La prima cosa che WCM non deve fare è saltare direttamente all'azione.

Tra richiesta ed esecuzione esiste un tratto di strada.

In forma compatta:

```

REQUEST
↓
INTENT
↓
GOAL + SCOPE
↓
AUTHORITY
↓
RESUME CHECK
↓
RUN / CHANGE
↓
CAPABILITY
↓
PROCESS + PROTOCOL ROUTING
↓
KNOWLEDGE RETRIEVAL
↓
CONTEXT SUFFICIENCY
↓
EXECUTION
↓
TRUE STOP / COMPLETION
↓
CONSOLIDATION

```

Questa figura è volutamente lineare per essere leggibile.

Il comportamento reale può contenere ritorni e verifiche.

Per esempio, durante il retrieval può emergere che l'authority non è quella inizialmente ipotizzata. Oppure una capability può fallire e richiedere una route alternativa. Oppure può apparire un workflow incompleto che ha priorità sul nuovo lavoro.

Il routing non è quindi una catena cieca.

È un percorso controllato verso un'azione autorizzata.

13.2 Comprendere l'intenzione

Una richiesta contiene parole.

WCM deve ricostruire l'intenzione operativa senza trasformare l'interpretazione in authority.

Consideriamo:

| «*Sistema il report.*»

La frase potrebbe significare:

- correggere un errore formale;
- aggiornare dati;
- modificare una conclusione;
- cambiare il modello con cui il report viene prodotto;
- rigenerare un output già previsto.

Sono attività molto diverse.

Per questo il primo passaggio è capire **che risultato sembra essere richiesto**.

Ma questa comprensione resta inizialmente un'ipotesi di routing.

INTERPRETAZIONE DELLA RICHIESTA

≠

AUTHORITY A ESEGUIRE QUALSIASI COSA CHE SEMBRA COERENTE

Se il significato necessario all'azione è già chiaro dalle fonti e dal contesto, WCM può procedere.

Se manca un'informazione indispensabile e non può essere recuperata dalle fonti autorevoli, può emergere una vera necessità di chiarimento o escalation.

13.3 Identificare goal e scope

Una richiesta utile deve essere collegata a un goal.

Il goal risponde:

| «**Quale risultato stiamo cercando di ottenere?**»

Lo scope risponde:

| «**Fin dove siamo autorizzati ad arrivare?**»

I due concetti sono vicini ma non equivalenti.

Esempio astratto:

GOAL

correggere un output incoerente

SCOPE

solo output e projection

FUORI SCOPE

modificare il protocollo che genera l'output

Questa distinzione impedisce un errore tipico dei sistemi molto autonomi: risolvere il problema visibile modificando qualcosa che appartiene a un livello superiore.

WCM cerca quindi goal e scope nelle fonti appropriate, usando la Source Precedence vista nel Capitolo 12.

13.4 Identificare authority

Capire cosa sarebbe utile fare non significa ancora poterlo fare.

WCM deve sapere quale authority possiede.

L'authority può derivare, a seconda del task, da:

- governance;
- baseline canonica;
- workflow già approvato;
- specific contract;
- authority receipt valido;
- istruzione owner che rientra in un perimetro già consentito.

La domanda è:

«Questa specifica transizione è già autorizzata?»

Non:

«In generale sarebbe una buona idea?»

Un principio fondamentale è che l'authority non viene ampliata per inferenza.

AUTHORITY A

≠

AUTHORITY A + QUALSIASI EFFETTO UTILE COLLEGATO

Un comando specifico può autorizzare una transizione precisa senza autorizzare una revisione del metodo.

13.5 Working Memory disponibile

Prima di aprire documenti WCM può avere Working Memory pertinente.

Questa memoria può contenere:

- il contesto della richiesta appena ricevuta;
- informazioni già verificate nella stessa attività;
- decisioni conversazionali recenti;
- risultati intermedi non ancora consolidati;
- riferimenti utili per sapere dove cercare.

La Working Memory evita di ripartire artificialmente da zero.

Ma resta valida la regola:

| Memory is not authority.

La Working Memory accelera il routing.

Non sostituisce la verifica persistente quando il task dipende da stato, canone, authority o workflow durevole.

13.6 Workflow da riprendere

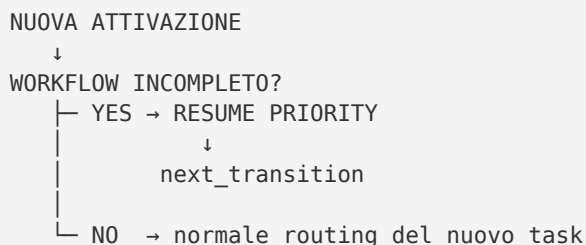
Questo è uno dei punti più importanti del routing WCM corrente.

Prima di cercare nuovo lavoro, il bootstrap deve verificare se esiste un workflow materiale incompleto.

Se esiste un workflow:

- **ACTIVE** con true stop non raggiunta; oppure
- **INTERRUPTED_RESUMABLE**;

si applica **Resume Priority**.



Il principio è semplice:

| la fine di una sessione non è la fine del workflow.

Il sistema non deve ricominciare il ragionamento da capo, né rieseguire step già completati, solo perché è arrivata una nuova attivazione.

Il checkpoint persistente serve proprio a trasformare la continuità da ricordo conversazionale a proprietà dell'esecuzione.

Se invece il workflow è **WAITING_AUTHORITY**, Resume Priority non autorizza a superare il gate.

Lo stato stesso rappresenta una vera stop condition finché l'authority richiesta non è disponibile.

13.7 RUN / CHANGE

Dopo aver identificato goal, scope e authority, WCM deve classificare la natura dell'azione.

La distinzione fondamentale è:

```
WCM RUN
=
esecuzione conforme a regole e workflow già autorizzati

WCM CHANGE
=
modifica materiale a metodo, governance, authority, scope, canone o baseline
```

Una scrittura persistente non è automaticamente un CHANGE.

Creare un draft previsto, aggiornare uno stato per riflettere un fatto realmente avvenuto o produrre una review già richiesta dal workflow possono essere normali RUN.

Al contrario, anche una modifica testuale piccola può essere un CHANGE se altera il significato normativo di un protocollo.

La classificazione deve quindi chiedere:

«Quale nodo materiale cambierebbe?»

Se cambia una regola del metodo, un gate, un'authority, uno scope o un'altra baseline materiale, entra in gioco il Change Gate.

Nel WCM corrente:

```
WCM CHANGE
→ Impact Preview
→ STOP
→ authority owner esplicita successiva
→ implementazione
```

Il routing deve intercettare questo confine **prima** della scrittura materiale non autorizzata.

13.8 Capability necessarie

Sapere cosa fare e poterlo fare sono due cose diverse.

WCM distingue la route della capability.

Il bootstrap generale espone quattro classi:

```
DIRECT
LOCAL_REQUIRED
SERVICE_REQUIRED
CAPABILITY_GAP
```

DIRECT

L'azione può essere svolta direttamente con le capability disponibili nella run corrente.

LOCAL_REQUIRED

Serve una capability locale o un ambiente specifico previsto dall'architettura.

SERVICE_REQUIRED

Il lavoro richiede un service autorizzato.

CAPABILITY_GAP

La capability necessaria non è realmente disponibile attraverso le route consentite.

Ma WCM applica una cautela importante:

un fallimento temporaneo non è automaticamente un capability gap.

Prima di concludere «non posso», la baseline richiede di verificare l'evidenza della capability e le alternative dirette previste.

Questo evita che un limite contingente venga trasformato in una proprietà permanente del sistema.

13.9 Processi applicabili

Una volta definito il tipo di lavoro, WCM deve capire **quale processo governa il percorso**.

Un processo descrive una sequenza organizzata di attività verso un risultato.

Esempi astratti di domande di routing:

- sto facendo bootstrap di contesto?
- sto consolidando memoria?
- sto promuovendo evidence verso una baseline?
- sto verificando integrità della conoscenza?
- sto aggiornando documentazione human-facing?
- sto riconciliando stato strutturato e viste derivate?

La richiesta non deve necessariamente nominare il processo.

Il routing collega l'intenzione al processo applicabile attraverso gli indici e le relazioni della Method KB.

Ma non inventa un processo nuovo perché la richiesta sembra insolita.

Se nessun processo esistente governa materialmente il caso, questo può diventare evidence di un gap metodologico. Non autorizza una modifica implicita del Process Book.

13.10 Protocolli applicabili

Il processo dice **come si sviluppa il lavoro**.

I protocolli introducono regole, guard, trigger e vincoli che possono attraversare più processi.

Per esempio, durante un normale task possono diventare applicabili protocolli relativi a:

- INDEX-FIRST;
- continuità del workflow;
- capability verification;
- authority command;
- knowledge health;
- persistent mutation safety;
- documentation impact;
- change closure.

Il punto importante è che non si caricano automaticamente tutti i protocolli.

Il routing corrente usa anche una sorgente machine-readable per eventi operativi:

`wcm/runtime/protocol-routing/ROUTING_SOURCE.json`.

Questa sorgente associa **eventi e hook esatti** ai processi/protocolli da caricare.

Esempio concettuale:

```
EVENTO
TOOL_OUTPUT_LIMIT

HOOK
ON_TOOL_FAILURE

ROUTE
PROT-011 + PROT-003 + PROT-009

AZIONE
RESOLVE_CAPABILITY
```

Questo meccanismo non sostituisce il reasoning cognitivo generale.

Riduce invece l'ambiguità quando il sistema ha già riconosciuto un evento operativo strutturato.

Il Capitolo 14 entrerà nel dettaglio di questo livello.

13.11 Recuperare i nodi necessari

A questo punto WCM sa abbastanza per iniziare il retrieval mirato.

Qui rientrano i meccanismi dei Capitoli 9-12:

```
ENTRY POINT
→ INDEX / MAP
→ ACTIVE AUTHORITY / PROCEDURE
→ EVIDENCE / HISTORY / RAW solo se necessario
```

Il routing e INDEX-FIRST lavorano insieme.

Il routing dice:

| *«Che cosa devo sapere per questa richiesta?»*

INDEX-FIRST dice:

| *«Qual è il percorso minimo e autorevole per saperlo?»*

Source Precedence aggiunge:

| *«Se trovo più fonti, quale deve prevalere per questa informazione?»*

Queste tre discipline non sono concorrenti.

Sono strati dello stesso movimento.

13.12 Context Sufficiency Gate

Il retrieval non termina quando «abbiamo letto abbastanza documenti».

Termina quando il contesto è sufficiente per agire in sicurezza.

PROC-005 definisce un Context Sufficiency Gate.

Prima dell'esecuzione l'agente deve sapere, per quanto rilevante al task:

- ruolo;
- progetto o goal;
- eventuale workflow da riprendere;
- contesto affidabile già disponibile;
- fonte persistente autorevole;
- authority;
- scope;
- next transition;
- processi e protocolli applicabili;
- azioni che richiedono escalation;
- true stop condition.

Quando queste risposte sono disponibili, altre letture devono essere motivate.

```
CONTESTO SUFFICIENTE
≠
CONTESTO MASSIMO
```

Questo gate protegge sia dall'ignoranza sia dall'over-retrieval.

13.13 Execution

Solo dopo il routing il sistema entra nell'esecuzione vera e propria.

Ma anche qui WCM non tratta l'azione come un punto isolato.

Se il workflow contiene più transizioni consecutive che sono:

- già previste;
- già autorizzate;
- classificabili come WCM RUN;
- non separate da una vera stop condition;

PROT-009 richiede **Contiguous Workflow Execution**.

```
TRANSIZIONE COMPLETATA
↓
ESISTE NEXT TRANSITION?
↓ YES
È GIÀ AUTORIZZATA?
↓ YES
È WCM RUN?
↓ YES
C'È UNA TRUE STOP QUI?
↓ NO
CONTINUE
```

Questo evita che il sistema scambi ogni output intermedio per una conclusione.

Una review completata, un file generato o la fine di una singola risposta non sono automaticamente la fine del workflow.

13.14 Checkpoint e continuità

Dopo una transizione materiale, il checkpoint deve essere aggiornato.

La logica è:

```
STEP A COMPLETATO
→ checkpoint: last=A / next=B
→ eventuale interruzione
→ nuova run legge next=B
→ ripresa senza duplicare A
```

Questa disciplina trasforma il routing in un sistema session-independent.

La nuova run non deve dedurre dove eravamo dalla prosa dell'ultima conversazione.

Lo legge dallo stato strutturato.

Per gli execution facts la precedence è:

```
AUTHORITY / CANON
→ runtime workflow checkpoint
→ derived state
→ human view
→ projection
→ UI
```

Il runtime dice dove si trova l'esecuzione.

Non crea nuova authority.

13.15 Stop condition

Un sistema autonomo deve sapere continuare.

Deve anche sapere fermarsi.

Le stop condition valide includono, secondo PROT-009 e la governance applicabile:

- gate definito dal workflow;
- decisione riservata a owner o authority competente;
- prossima transizione classificata WCM CHANGE senza authority;
- blocker reale;
- capability gap verificato;
- errore tecnico che impedisce prosecuzione sicura;
- limite tecnico reale della run;
- azione sensibile o irreversibile soggetta a escalation.

Una stop condition non è un fallimento per definizione.

WAITING_AUTHORITY, per esempio, può essere lo stato perfettamente corretto di un workflow che ha raggiunto il suo gate umano.

```
FERMARSI DOVE PREVISTO
=
COMPORTAMENTO CORRETTO
```

13.16 Completion Gate

Anche la parola «completato» ha un significato preciso.

Non basta avere prodotto qualcosa.

Prima di dichiarare un workflow **COMPLETED**, WCM deve verificare che i requisiti di completion siano realmente soddisfatti.

Tra questi, in funzione del workflow:

- output completi;
- true stop raggiunta;
- checkpoint corrente;
- execution view coerente;
- impact set propagato;
- current-facing mirrors coerenti;
- assurance risolta;
- next eligibility risolta.

Il principio è:

```
OUTPUT  PRODOTTO
≠
WORKFLOW COMPLETATO
```

Questa distinzione è essenziale per evitare falsi successi.

13.17 Consolidation

Dopo un delta materiale il lavoro non finisce necessariamente con l'output.

Può essere necessario consolidare ciò che è cambiato nella Persistent Organizational Memory.

PROC-006 governa questo passaggio.

La consolidation non significa salvare tutta la conversazione.

Significa persistere il delta organizzativamente rilevante e riallineare i nodi che devono rifletterlo.

Il routing completo termina quindi con una domanda:

«Che cosa deve sopravvivere a questa run?»

Se la risposta è «nulla di materiale», non si inventa memoria.

Se esiste un delta materiale, deve essere consolidato nel posto giusto.

13.18 Il routing non è un prompt gigantesco

A questo punto potrebbe sembrare che, per ogni richiesta, WCM debba caricare governance, tutti i processi, tutti i protocolli, tutti i runtime e tutta la KB.

Sarebbe l'opposto del modello.

Il routing serve proprio a evitare questo.

REQUEST

- identifica ciò che serve
- carica solo i nodi applicabili
- stop when sufficient

Il sistema non porta tutta l'organizzazione dentro ogni task.

Porta nel task **la parte dell'organizzazione necessaria a quel task**.

13.19 Cognitivo e deterministico

Il Request Routing contiene due nature diverse.

Parte cognitiva

Serve per:

- comprendere l'intenzione;
- collegare una richiesta non strutturata a goal e scope;
- riconoscere possibili conflitti di significato;
- valutare se il contesto è semanticamente sufficiente;
- interpretare documenti human-facing quando non esiste un campo strutturato equivalente.

Parte deterministica

Quando esiste una primitive strutturata, WCM evita di sostituirla con interpretazione probabilistica.

Esempi:

- stato del workflow da checkpoint JSON;
- derived state da runtime validato;
- event routing da exact event + exact hook;
- verifica di schema;
- mapping di projection;
- telemetry materialization.

REASONING

- dove serve significato

DETERMINISTIC PRIMITIVE

- dove il significato è già stato strutturato

WCM non cerca di rendere deterministica ogni attività cognitiva.

Cerca di non usare cognizione probabilistica per operazioni che possono essere meccaniche.

13.20 Un esempio astratto completo

Richiesta:

| «Continua il lavoro e porta l'output al prossimo gate.»

1. Intent

Continuare un workflow esistente, non inventare un nuovo goal.

2. Goal e scope

Recuperati dall'entry point e dal workflow persistente.

3. Authority

Verificare che il workflow e le transizioni successive siano già autorizzati.

4. Resume check

Il runtime mostra:

```
status = INTERRUPTED_RESUMABLE
next_transition = REVIEW
```

Si applica Resume Priority.

5. RUN / CHANGE

REVIEW è una transizione già prevista: WCM RUN.

6. Capability

La review è eseguibile direttamente: **DIRECT**.

7. Process / Protocol

Bootstrap + INDEX-FIRST + Contiguous Workflow Execution.

8. Retrieval

Si leggono soltanto il checkpoint, la baseline da revisionare e le regole necessarie alla review.

9. Sufficiency

Authority, input, next transition e true stop sono chiari.

10. Execution

La review viene eseguita.

Se il workflow prevede una revisione successiva già autorizzata e nessuna stop condition, si continua.

11. Stop

Il workflow raggiunge un gate owner.

Lo stato corretto diventa **WAITING_AUTHORITY**.

Il sistema si ferma.

Non perché non sappia cosa fare dopo.

Ma perché **non possiede ancora l'authority per farlo**.

13.21 Un secondo esempio: richiesta che nasconde un CHANGE

Richiesta:

❗ *«Fai in modo che da oggi questo controllo non sia più necessario.»*

L'azione potrebbe sembrare una semplice semplificazione.

Il routing verifica però che quel controllo sia imposto da un protocollo ACTIVE.

La richiesta modifica quindi una baseline materiale.

```
INTENT
rimuovere un controllo

SOURCE CHECK
controllo imposto da protocollo ACTIVE

CLASSIFICATION
WCM CHANGE

ROUTE
Impact Preview
→ STOP
→ authority owner esplicita
```

Il routing ha impedito che una frase operativa diventasse una modifica silenziosa del metodo.

13.22 Un terzo esempio: capability failure

Richiesta:

❗ *«Recupera il dato e completa il report.»*

Durante l'esecuzione uno strumento restituisce un output limitato.

Un sistema ingenuo potrebbe concludere:

❗ *«Capability non disponibile.»*

Il routing machine-readable corrente riconosce invece l'evento **TOOL_OUTPUT_LIMIT** sull'hook **ON_TOOL_FAILURE** e carica la route prevista per verificare alternative e continuità.

Il significato è importante:

```
PRIMO FALLIMENTO
≠
CAPABILITY GAP
```

Soltanto dopo la verifica prevista può emergere un vero blocker tecnico.

13.23 Anti-pattern del Request Routing

Anti-pattern 1 — Request → Action

Saltare goal, scope e authority perché la richiesta sembra chiara.

Anti-pattern 2 — Nuovo task prima del resume

Ignorare un workflow incompleto e ricominciare lavoro già avviato.

Anti-pattern 3 — Tutto è CHANGE

Classificare ogni write come modifica del metodo.

Anti-pattern 4 — Nulla è CHANGE

Trattare una modifica normativa come semplice aggiornamento documentale.

Anti-pattern 5 — Carico tutti i protocolli

Confondere sicurezza con over-retrieval.

Anti-pattern 6 — Tool failure = impossibile

Dichiarare un capability gap senza evidence check.

Anti-pattern 7 — Output = completion

Terminare appena è stato prodotto un artefatto, anche se il workflow prevede altre transizioni contigue.

Anti-pattern 8 — UI = stato esecutivo master

Usare una projection al posto del runtime strutturato quando il fatto esecutivo è disponibile nel checkpoint.

13.24 La formula compatta

Possiamo condensare il Request Routing WCM in una domanda composta:

Che cosa mi viene chiesto, quale risultato devo ottenere, entro quale scope e authority, da quale stato devo partire, quali regole e capability si applicano, quale contesto minimo mi serve e dove devo fermarmi?

In forma strutturale:

```
REQUEST
+
INTENT
+
GOAL / SCOPE
```

```
+  
AUTHORITY  
+  
EXECUTION STATE  
+  
RUN / CHANGE  
+  
CAPABILITY  
+  
PROCESS / PROTOCOL  
+  
MINIMUM AUTHORITATIVE CONTEXT  
+  
TRUE STOP  
=  
ROUTED WORK
```

13.25 Cosa abbiamo ottenuto

Con il Request Routing il Knowledge Navigation Layer entra nel ciclo operativo.

Nei capitoli precedenti avevamo costruito:

```
COME TROVARE  
+  
COSA LEGGERE  
+  
QUALE FONTE PREVALE
```

Ora aggiungiamo:

```
PERCHÉ STIAMO CERCANDO  
+  
QUALI REGOLE SI APPLICANO  
+  
SE POSSIAMO AGIRE  
+  
DOVE RIPRENDERE  
+  
DOVE FERMARCI
```

Il risultato è un passaggio fondamentale:

```
RICHIESTA UMANA O OPERATIVA  
↓  
CONTESTO GOVERNATO  
↓  
LAVORO ESEGUIBILE
```

Nel Capitolo 14 entreremo nel punto più specifico della route:

come WCM individua i protocolli da applicare senza caricare indiscriminatamente l'intero Protocol Book.

Source Map

Fonti canoniche principali

- [WCM_AGENT_START.md](#) — bootstrap, source precedence, RUN/CHANGE, capability routing, stop condition e Completion Gate;
- [wcm/GOVERNANCE.md](#) — autonomia, RUN vs CHANGE, execution horizon, authority e stop riservati;
- [wcm/kb/index.md](#) — Method KB entry point e route verso processi/protocolli;
- [wcm/process-book/processes/PROC-005_AGENT_READY_CONTEXT_BOOTSTRAP.md](#) — bootstrap, Resume Priority e Context Sufficiency Gate;
- [wcm/process-book/protocols/PROT-005_INDEX_FIRST_PROGRESSIVE_RETRIEVAL.md](#) — retrieval progressivo, Source Precedence e Stop When Sufficient;
- [wcm/process-book/protocols/PROT-009_CONTIGUOUS_WORKFLOW_EXECUTION.md](#) — Resume Priority, execution contigua, checkpoint, stop condition e Completion Gate;
- [wcm/runtime/protocol-routing/ROUTING_SOURCE.json](#) — exact event + exact hook routing corrente per eventi operativi strutturati.

Relazioni

```
CH13
├ CONTINUES → CH09 / CH10 / CH11 / CH12
├ EXPLAINS → PROC-005
├ GOVERNED_BY → GOVERNANCE
├ GOVERNED_BY → PROT-005
├ GOVERNED_BY → PROT-009
├ REFERENCES → ROUTING_SOURCE.json
└ PREPARES → CH14
```

Maturity note

Il Request Routing descritto qui rappresenta la baseline WCM corrente. Alcune componenti sono cognitive, altre sono supportate da primitive deterministiche già implementate. La field validation del modello complessivo continua: il capitolo non implica che ogni richiesta in ogni dominio sia già instradabile senza ambiguità, né che la comprensione semantica sia diventata deterministica.